

复习题

Part1: 填空题

1. 并行化是现代计算机性能提升的重要途径，实现并行化可以通过以下三种方式：
_____、_____和_____。

答案：指令并行（ILP）、数据并行（DLP）、线程并行（TLP）

2. _____编程方式适合在单机内并行；_____编程方式适合在多机之间并行。

答案：OpenMP、MPI

3. HPC Top 500 的排名依据是_____。

答案：High Performance Linpack（HPL）测试结果

4. 配置一套 200TFlops 的 HPC 集群，如果使用双路 Intel X5670 处理器来构建的话，总共需要_____个节点。（Intel X5670 为 2.93GHz 六核处理器，每时钟周期可执行 4 个双精度浮点计算）

答案：节点数=200TFlops/（2CPU×2.93GHz×6Core×4Flops/Hz）=1422

5. InfiniBand 的高性能主要体现在：_____、_____和_____。

答案：高吞吐量、低延迟、低 CPU 负载

6. 华为完全自研的 HPC 集群调度器命名为_____。

答案：Donau（Donau 来自德语“多瑙河”）

7. HPC 的典型应用场景包括：_____、_____、_____、_____、_____等。

答案：气象科学、能源勘探、航空航天、生命科学、分子动力学、卫星测绘、基础科学研究、汽车电子

Part2: 单选题

1. 下列不属于 GPU 典型应用场景的是？（ ）

- A. 视频图像处理场景
- B. 通用计算，完成复杂逻辑运算
- C. HPC 高性能加速场景
- D. HPC 混合应用高吞吐量场景

答案：B

2. 服务器集群具有很强的可伸缩性，随着需求和负荷的增长，一般可通过以下哪种方式提高集群的性能？（ ）

- A. 增加集群中的网络带宽
- B. 更换集群中单服务器的硬盘为高 I/O 的固态硬盘
- C. 增加集群中单服务器的内存容量
- D. 向集群系统中添加新的集群服务器

答案：D

3. 集群是一组协同工作的服务器实体，通过提高单位时间内执行的任务数来提升效率。（ ）

- A. TRUE
- B. FALSE

答案：A

4. 以下关于 HPC 高性能计算的说法错误的是？（ ）

- A. HPC 计算集群主要由计算服务器、共享存储、高速互连设备以及配套的管理软件、平台软件和应用软件构成
- B. HPC 即高性能计算，指以计算为目的，使用了很多处理器的单个计算机系统或使用了多台计算机集群的计算系统和环境
- C. HPC 就是超级计算机，是用计算机去研究、设计产品及支持复杂的决策

D. 早期的 HPC 基于定制计算机，专门用于基础科学研究和国防战略等特定领域，随着技术的发展，现在绝大部分 HPC 基于 X86 服务器的集群架构进行构建

答案：C

5. 异构计算是继单核计算，多核并行计算之后新的计算模型，下列关于异构计算的背景描述错误的是？（ ）

A. 多核并行计算带来的性能提升是有限的，随着并行度的提升，散热和能耗的问题也日益突出

B. 异构计算能够最大化的发挥各处理器的性能，把最适合的任务交给最擅长的计算单元来完成

C. 摩尔定律越来越接近物理极限，提升单核性能的难度和成本越来越高

D. NUMA 是异构计算的一种模式

答案：D

6. GPU 因为 SIMD（单指令流多数据流，Single Instruction Multiple Data）架构，在机器学习领域上被广泛应用，最近 FPGA 也开始流行起来。下列关于 FPGA 和 GPU 的说法正确的是？（ ）

A. GPU 的优势是，如果计算机需要改变，它可以被重新装配

B. 就峰值性能来说，GPU 低于 FPGA

C. FPGA 可以根据特定的应用编程硬件，灵活性远好于 GPU

D. GPU 执行单元少，只能单线程处理指令

答案：C

7. 下面哪项不属于集群系统的主要优点？（ ）

A. 高可用性

B. 高兼容性

C. 高性能

D. 高可扩展性

答案：B

8. InfiniBand 是一个用于高性能计算的计算机网络通信标准，它具有极高的（ ）和极低的（ ）。

A. 吞吐量、延迟

B. 传输速率、错误率

C. 吞吐量、错误率

答案：A

9. 以下关于服务器集群管理特性的描述中，说法不正确的是？（ ）

A. 服务器集群都可以用 WebUI 进行管理

B. 特殊应用场景的集群软件有独立的管理方式

C. 独立的服务器集群在管理层面上相当于一个单机系统

D. 系统管理员可以从远程管理一个甚至一组集群，就好像在单机系统中一样

答案：A

10. 当前设计高性能计算机的重要技术途径是？（ ）

A. 提高 CPU 主频

B. 扩大内存容量

C. 采用非冯·诺依曼的结构

D. 采用并行处理技术

答案：D

Part3: 多选题

1. 高性能计算当前面临的主要挑战是？（ ）

- A. 算力需求持续增长
- B. 行业应用多样化，对系统性能要求各异
- C. 大规模计算集群带来高昂的能耗支出
- D. 摩尔定律遭遇瓶颈，性能提升放缓
- E. 使用、部署和管理门槛高，需要专业的技术能力

答案：A、B、C、E

2. 以下关于异构计算特点的理解中正确的是？（ ）

- A. 异构计算的目的是为了了解决 CPU 计算能力不足的问题
- B. 异构计算就是制定出一系列软件与硬件的标准，让不同类型的计算设备能够共享计算的过程和结果
- C. 常见的异构计算系统包括，CPU+GPU、GPU+FPGA、CPU+专用芯片
- D. GPU 和 CPU 协同工作可组成异构计算系统，异构计算的优势为计算加速和节能

答案：A、B、D

3. 通常情况下，提高性能最常见的方法有？（ ）

- A. 购买更快的硬件
- B. 修改算法
- C. 修改数据结构
- D. 修改执行
- E. 采用并发机制

答案：A、B、C、E

Part4: 简答题

1. 有一套配置 256 个双路 Intel X5670 处理器计算节点的 HPC 集群，试计算该 HPC 集群的峰值计算能力。（Intel X5670 为 2.93GHz 六核处理器，每时钟周期可执行 4 个双精度浮点计算）

答：峰值计算性能 = $2.93\text{GHz} \times 4\text{Flops/Hz} \times 6\text{Core} \times 2\text{CPU} \times 256 \text{ 节点}$
 $= 36003.8\text{GFlops} = 36.003\text{TFlops}$ 。即 36.003 万亿次每秒的峰值计算性能。

2. 请简述 RDMA（Remote Direct Memory Access）是如何实现的。

答：数据发送/接收时，不用将数据拷贝到缓冲区中，而是直接将数据发送到对方。绕过系统核心，实现零拷贝。

3. 当前主流 CPU 通常采用哪几种技术来实现并发机制？

答：四种，流水线、多发射、MPI、异构计算

流水线技术（Pipeline）——如果一个处理器要处理若干个任务，且每个任务由相同的 n 个步骤组成，不同的步骤又可同时执行，则处理器可采取流水线方式处理这些任务。流水线的目的是获得更高的吞吐量。当任务数很多时，系统吞吐量接近于 $1 / \text{最大子任务持续时间}$ 。

多发射——是一种每个时钟周期可同时发出多条指令供处理器并行执行的指令级并行技术。时间上的并行采用流水线技术，而空间上的并行则采用多发射。

MPI——Message Passing Interface 也就是“消息传递接口”。它是一个信息传递应用程序接口，包括协议和语义说明，MPI 的目的是高性能、大规模、和可移植性。MPI 是高性能计算的主要模型。

异构计算——异构计算是一种特殊形式的并行和分布式计算，它或者使用能同时支持 SIMD 方式和 MIMD 方式的单个独立计算机，或者使用由高速网络互连的一组独立计算机来完成计算任务。它能协调地使用性能、结构各异的计算设备以满足不同的计算需求，并使代码（或代码段）能以最大总体性能方式来执行。

4. 设计并行程序，目标使并行加速度达到最大的主要步骤有哪些？

答：1) 分解任务，确定可并行执行的部分，将大任务分解成多个可并行执行的小任务。尽量使所有处理器都有任务处理，但是也要避免太多太小的任务。
 2) 分配任务，将每个任务分配到线程中。尽量保证负载均衡。
 3) 指挥控制，主要是管理数据和协调多个线程之间的通信。尽量保证线程执行的正确性，减少串行化部分和同步开销。
 4) 映射，将线程映射到处理器上执行。

5. 编写多线程代码时要注意的问题主要包括哪些？

答：1) 负载均衡；
 2) 正确的存取共享变量（通过互斥代码或互斥锁实现）。

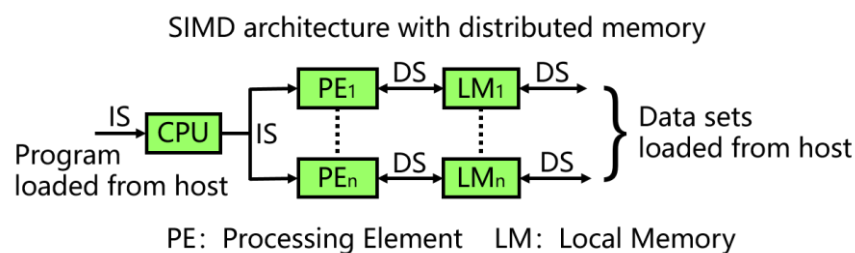
6. 集群技术的优势有哪些？

答：1) 通用的高性能：节点采用传统服务器平台，通用的硬件和操作系统，适应性强；
 2) 高可用性：高度的设备冗余，CPU、内存、磁盘、节点机
 3) 高可扩展性：以交换设备为核心，节点机、存储可灵活增减
 4) 更高的性价比：通用设备，统一的标准

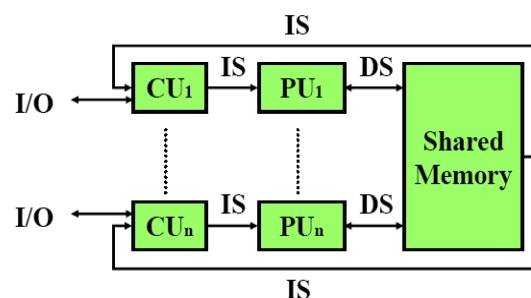
Part5: 名词解释

解释以下基本概念：

1. HPC——（High Performance Computing）高性能计算，也可以称为并行计算。在并行计算机或分布式计算机等高性能计算系统上所进行的超级计算。主要通过集群架构、并行算法和相关软件以并行/分布式计算的方式实现单台计算机无法达到的运算速度。
2. HPCC——（High Performance Computing and Communication）高性能计算与通信。指分布式高性能计算、高速网络以及高速 Internet 访问。
3. Distributed computing——分布式计算。一个计算任务由多台计算机共同完成，由传统的人和软件之间的交互变成软件和软件之间的数据交互。
4. Grid computing——网格计算。利用互联网把分散在不同地理位置的电脑组织成为一个“虚拟的超级计算机”，其中每一台参与计算的计算机就是一个“节点”，而整个计算是由成千上万个“节点”组成的“一张网格”。
5. SIMD——单指令多数据流。所有“活动的”处理器在同一时刻执行同一条指令对多个数据流进行操作，如下图所示。



MIMD: 多指令多数据流。每台处理机执行自己的指令，操作数也是各取各的。



MIMD architecture with shared memory

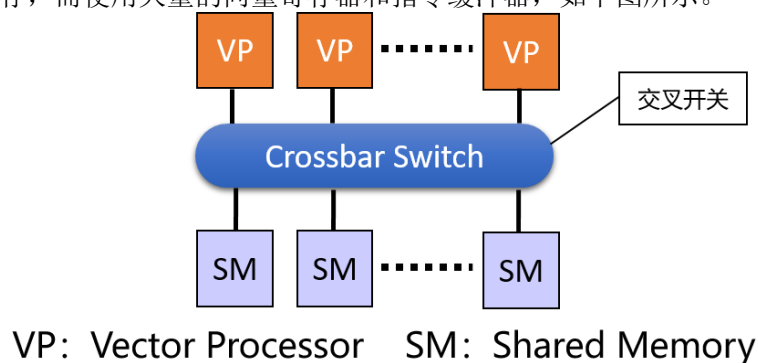
SISD: 单指令单数据流。传统的串行处理机。CPU执行单一的指令流对单一的数据流进行操作

作。

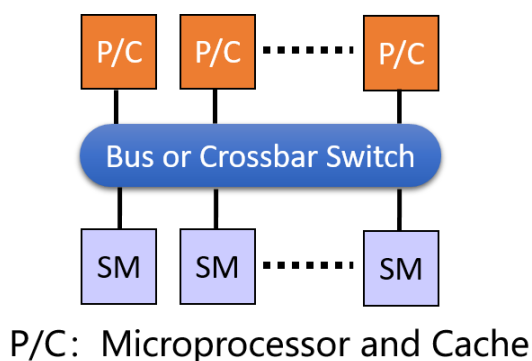
IS : Instruction Stream DS : Data Stream
CU : Control Unit PU : Processing Unit
MU : Memory Unit



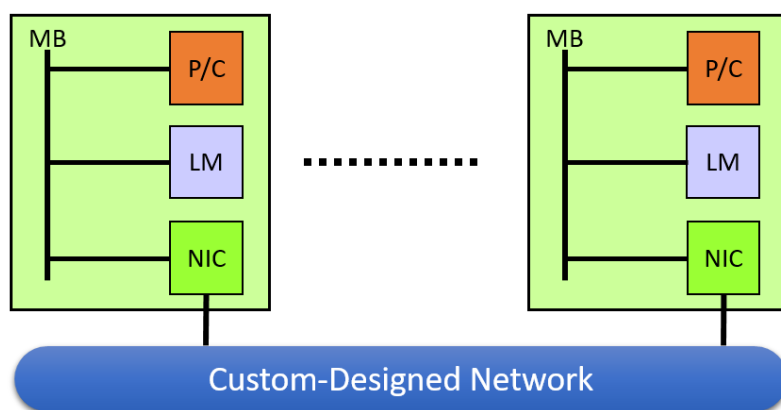
6. PVP—— 并行向量处理器 (Parallel Vector Processor)。系统中包含为数不多的高性能特制的向量处理器，使用专门设计的高带宽交叉开关网络将向量处理器连向共享存储模块。通常不使用高速缓存，而使用大量的向量寄存器和指令缓冲器，如下图所示。



7. SMP——对称多处理 (Symmetric Multiprocessor)。节点包含两个或两个以上完全相同的处理器，在处理上没有主 / 从之分。每个处理器对节点计算资源享有同等访问权。SMP 系统使用商品微处理器 (具有片上或外置高速缓存)，它们经由高速总线或交叉开关，连向共享存储器，如下图所示。

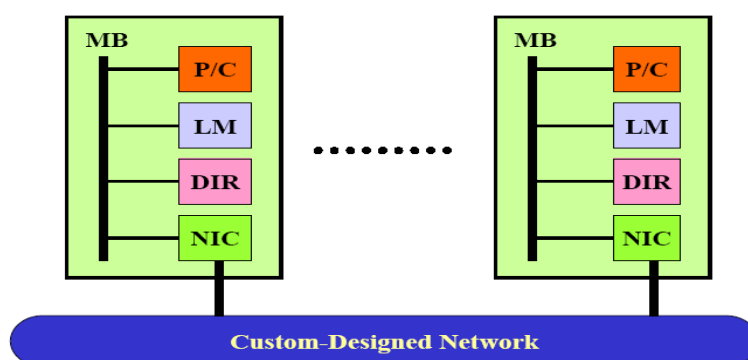


8. MPP——大规模并行处理机 (Massively Parallel Processor)。节点传统上是由单一 CPU、少量的内存、部分 I/O、节点间互联以及每个节点的操作系统组成的一个实例。节点间互联 (以及驻留于各节点的操作系统实例) 不需要硬件一致性，因为每个节点拥有其自己的操作系统以及自己唯一的物理内存地址空间。因而，一致性是在软件中通过“消息传送”(Message passing) 实现的。MPP 具有以下特性：1) 处理节点采用商用微处理器；2) 系统中有物理上的分布存储器；3) 采用高带宽低延迟的互连网络；4) 能扩展到上千至上万个处理器；5) 异步的 MIMD 机器，进程间采用传递消息相互作用，如下图所示。

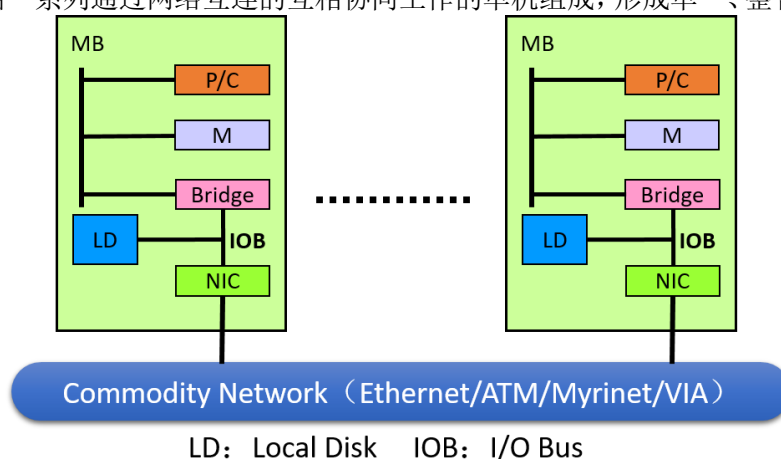


9. **DSM**: 分布共享存储多处理机。在物理上有分布在各节点的局部存储器，从而形成一个共享的存储器。对用户而言，系统硬件和软件提供了一个单地址的编程空间。

MB : Memory Bus NIC : Network Interface Circuitry
DIR : Cache Directory



10. **Cluster**——集群。系统中的每个节点拥有小于 16 个处理器。**Cluster** 是一种并行或分布式处理系统，由一系列通过网络互连的互相协同工作的单机组成，形成单一、整合的计算资源。



LD: Local Disk IOB: I/O Bus

11. **UMA**——统一内存访问模型 (Uniform Memory Access)。UMA 具有以下特点：1) 物理存储器被所有处理器均匀共享，所有处理器访问任何存储单元花费相同的时间；2) 每台处理器可带私有高速缓存；3) 外围设备也可以一定形式共享。

12. **NUMA**——非统一内存访问模型 (Non-uniform Memory Access)。NUMA 具有以下特点：1) 被共享的存储器在物理上是分布在所有的处理器中的，组成全局地址空间；2) 处理器访问存储器的时间是不同的，访问本地存储器或群内共享存储器较快，访问外地存储器或全局存

存储器较慢；3）每台处理器可带私有高速缓存，外围设备也可以一定形式共享。

13. CC-NUMA——高速缓存一致性非统一内存访问模型（Coherent-cache Non-uniform Memory Access）。实际上是将一些 SMP 机器作为一个单节点并彼此连接起来，所形成的一个较大的系统。CC-NUMA 具有以下特点：1）使用基于目录的高速缓存一致性协议；2）保留 SMP 结构易于编程的优点，改善了常规 SMP 的可扩展性问题；3）实际上是一个分布共享存储的 DSM 多处理机系统；4）最显著的优点是程序员无需明确地在节点上分配数据，在运行时高速缓存一致性硬件会自动地将数据迁移到要用到的它的地方。

14. PUE——电力使用效率（Power Usage Effectiveness）。PUE = 数据中心总能耗/IT 设备能耗，PUE 越接近 1 表明非 IT 设备耗能越少，即能效水平越高。数据中心的冷却占总功耗的 40% 左右，机房冷却主要由空调负责，所以降低空调的耗电量可以有效的降低 PUE 的值。当然，要做到最佳节能，降低服务器等 IT 设备的功耗，才是最根本和有效的方法。

15. SVE——可伸缩向量扩展（Scalable Vector Extension）。SVE 是 ARMv8 AArch64 架构的新一代 SIMD 指令集，它不是 NEON 的替代，而是专门为高性能计算工作负载的向量化而开发的。SVE 引入了很多新的架构特点：1）可变矢量长度；2）每通道预测；3）聚集加载和分散存储；4）横向操作等。

Part6: 综合题

1、试比较 PVP、SMP、MPP、DSM 和 Cluster 并行机结构的不同点，以典型系统举例说明。

表 1.6 汇总了上述 5 种结构的特性比较。

属性	PVP	SMP	MPP	DSM	COW
结构类型	MIMD	MIMD	MIMD	MIMD	MIMD
处理器类型	专用定制	商用	商用	商用	商用
互连网络	定制交叉开关	总线、交叉开关	定制网络	定制网络	商用网络（以太 ATM）
通信机制	共享变量	共享变量	消息传递	共享变量	消息传递

地址空间	单地址空间	单地址空间	多地址空间	单地址空间	多地址空间
系统存储器	集中共享	集中共享	分布非共享	分布共享	分布非共享
访存模型	UMA	UMA	NORMA	NUMA	NORMA
代表机器	Cray C-90, Cray T-90, 银河 1 号	IBM R50 , SGI Power Challenge, 曙光 1 号	Intel Paragon, IBMSP2, 曙光 1000/2000	Stanford DASH, Cray T 3D	Berkeley NOW, Alpha Farm

表 1.6 5 种结构特性一览表

2、列出常用静态和动态网络的主要参数（节点度、直径、对剖带宽和链路数）以及复杂度、网络性能、扩展性和容错性等。

答：静态网络（Static Networks）是指处理单元间有着固定连接的一类网络，在程序执行期间，这种点到点的链接保持不变；动态网络（Dynamic Networks）是用交换开关构成的，可按应用程序的要求动态地改变连接组态。

典型的静态网络有一维线性阵列、二维网孔、树连接、超立方网络、立方环、洗牌交换网、蝶形网络等；典型的动态网络包括总线、交叉开关和多级互连网络等。

网络名称	网络规模	节点度	网络直径	对剖宽度	对称	链路数
线性阵列	N 个节点	2	$N - 1$	1	非	$N - 1$
环形	N 个节点	2	$\lfloor N/2 \rfloor$ (双向)	2	是	N
2-D 网孔	$(\sqrt{N} \times \sqrt{N})$ 个节点	4	$2(\sqrt{N} - 1)$	$\sqrt{N}$	非	$2(N - \sqrt{N})$
Illiac 网孔	$(\sqrt{N} \times \sqrt{N})$ 个节点	4	$\sqrt{N} - 1$	$2\sqrt{N}$	非	$2N$
2-D 环绕	$(\sqrt{N} \times \sqrt{N})$ 个节点	4	$2\lfloor \sqrt{N}/2 \rfloor$	$2\sqrt{N}$	是	$2N$

二叉树	N 个节点	3	$2(\lceil \log N \rceil - 1)$	1	非	$N - 1$
星形	N 个节点	$N - 1$	2	$\lfloor N/2 \rfloor$	非	$N - 1$
超立方	$N = 2^n$ 节点	n	n	$N/2$	是	$nN/2$
立方环	$N = k \cdot 2^k$ 节点	3	$2k - 1 + \lfloor k/2 \rfloor$	$N/(2k)$	是	$3N/2$

表 2.1 静态互连网络特性一览表

下面我们比较在可扩展计算机平台或计算机机群系统中为了实现系统动态互连，系统总线、多级网络和交叉开关的硬件需求和潜在的性能。

Networks	Delay	Cost	Blocking	Example
Bus	$O(N)$	$O(1)$	Yes	SunFire Gigaplane Bus: 2.67GB/s
MINs	$O(\log N)$	$O(N \log N)$	Yes	IBM SP2: 10.24GB/s
Crossbar	$O(1)$	$O(N^2)$	No	Digital Giga Switch: 3.4GB/s

3、比较并行计算模型 PRAM、BSP 和 logP。评述它们的差别、相对优点以及在模型化真实并行计算机和应用时的局限性。

1) PRAM模型：并行随机存取机器，也可称为共享存储的SIMD模型。

特点：假定存在着一个容量无限大的共享存储器，有有限或无限个功能相同的处理器，且均有简单的算术运算和逻辑判断功能。在任何时刻各处理器均可通过共享存储单元相互交换数据。

优点：特别适合于并行算法的表达、分析和比较；使用简单，很多诸如进程间通信、存储管理和进程同步等并行机的低级细节均隐含于模型中；易于设计和稍加修改便可运行在不同的并行机上；且有可能在PRAM模型中加入一些诸如同步和通信等需要考虑的问题。

缺点：PRAM是一个同步模型，这意味着所有指令均按锁步方式操作；用户虽感觉不到同步的存在，但它的确是很费时的；共享单一存储器的假定，显然不适合分布存储的异步的

MIMD机器：假定每个处理器均可在单位时间内访问任何存储单元而略去存取竞争和有限带宽等是不现实的。

2) **BSP模型：**“大”同步模型，是个分布存储的**MIMD**计算模型。

特点：**BSP**将处理器和选路器分开，强调了计算任务和通信任务的分开，而选路器仅施行点到点的消息传递，不提供组合、复制或广播等功能，这样做既掩盖了具体的互联网拓扑，又简化了通信协议；采用路障方式的以硬件实现的全局同步是在可控的粗粒度级，从而提供了执行紧耦合同步式并行算法的有效方式，而程序员无过分的负担；为**PRAM**模型所设计的算法均可采用在每个**BSP**处理器上模拟一些**PRAM**处理器的方法实现之。

- 优点

- 在软件和硬件之间架起一座类似于冯·诺依曼的桥梁；
- 如果计算和通信可合适的平衡，则可克服分布**MIMD**模型编程能力较差的特点；
- 实现了一些重要的算法，且均避免了自动存储管理的额外开销；
- 可有效地在超立方网络和光交叉开关互利技术上实现，与特定的工艺技术无关，只要选路器有一定的通信吞吐率。

- 缺点

- 在**BSP**模型中，要求超级步的长度必须能充分地适应任意的h-relation；
- 超级步发送的消息最快也要在下一个超级步才可以使用；
- **BSP**模型中的全局路障同步假定是用特殊的硬件支持，在很多并行机中可能没有相应的现成的硬件机构。

3) **logP模型：**是一种分布存储的，点到点通信的多处理机模型。

- 特点

logP模型是一种分布存储的、点到点通信的多处理机模型，其中通信网络由一组参数来描述，但它不涉及具体网络结构，也不假定算法一定要用显式的消息传递操作进行描述（实现隐式同步）。

- 优点

- **logP**模型将现代和将来的并行机的特性进行了精确的综合，以少量的参数L、o、g和p刻画了并行机的主要瓶颈；
- 无须说明编程风格或通信协议，可以等同地用于共享存储、消息传递和数据并行等各种风范；
- **logP**模型的可用性已得到多个算法的证实；
- **logP**模型是**BSP**模型的细化，也可以转换为**PRAM**模型；
- 打开了研究模型的新途径，为设计并行机体系结构提供了指导性意见。

- 缺点

- （难以进行算法描述、设计和分析）

BSP和logP相互比较：1. 现今最流行的并行计算模型是**BSP**和**logP**，已经证明两者本质上是等效的，且可以互相模拟；2. **BSP**为算法和程序提供了更多的方便，而**logP**却提供了较好的机器资源的控制；3. **BSP**所引起的精确度方面的损失比起其所提供的更结构化的编程风格的优点来是小的；4. **BSP**模型在简明性、性能的可预测性、可移植性和结构化可编程性等方面更受人欢迎和喜爱。

三者关系：

LogP和**PRAM**模型是并行计算模型的两个极端。**BSP**模型可以看成是上述两个模型的折衷。相比之下，**LogP**模型过于复杂，缺乏有效的分析和性能预测的模型，而**PRAM**则过于简单，无法

真实地描述物理机器。BSP模型较好地综合了其它两个模型优点,在面向物理机器实现方面优于PRAM模型,而和LogP模型相比,又更加便于进行算法设计和性能预测。

4、比较在 PRAM 模型和 BSP 模型上,计算两个 N 阶向量内积的算法及其复杂度。

PRAM 模型

求两个 N 维向量 A, B 的内积 $s (s = \sum a * b)$

串行:

N 个乘法, $N-1$ 个加法

共需 $2N$ 个周期

PRAM 机(n 个处理器):

每个处理器 p 完成 N/n 个乘法, $N/n-1$ 个加法,共 $2N/n$ 个周期,然后采用树归约方法将 n 个局部和相加--- $\log n$ 周期,共需要 $2N/n + \log n$ 个周期

加速比: $2N / (2N/n + \log n) \approx n (N \gg n)$

BSP 模型

求两个 N 维向量 A, B 的内积 $s (s = \sum a * b)$ 假设 8 个处理器

超步 1:

计算:每个处理器在 $w=2N/8$ 周期内计算局部和

通讯:处理器 0,2,4,6 将局部和送给 1,3,5,7

路障同步

超步 2:

计算:处理器 1,3,5,7 各自完成一次加法($w=1$)

通讯:1,5 将中间结果送给 3,7

路障同步

超步 3:

计算:处理器 3,7 各自完成一次加法($w=1$)

通讯:3 将中间结果送给 7

路障同步

超步 4:

计算:处理器 7 完成一次加法($w=1$),产生最后结果

总执行时间: $2N/8 + 3g + 3l + 3$ 个周期.在 n 个处理器的 BSP 机上,需 $2N/n + \log n (g+l+1)$ 个周期,比

PRAM 多了 $(g+l) * \log n$,其分别对应于通讯和同步的开销

5、什么是加速比 (speed up)、并行效率 (efficiency) 和可扩展性 (scalability)? 如何描述在不同约束下的加速比?

答: **粒度**: 是各个处理机可独立并行执行的任务大小的度量 大粒度反映可并行执行的运算量大,亦称为粗粒度 指令级并行等则是小粒度并行,亦称为细粒度

加速比: 串行执行时间为 T_s , 使用 q 个处理机并行执行的时间为 $T_p(q)$, 则加速比为 $Sp(q) = T_s / T_p(q)$ 。

简单的说,并行系统的加速比是指对于一个给定的应用,并行算法(或并行程序)的执行速度相对于串行算法(或串行程序)的执行速度快了多少倍。

效率: 设 q 个处理机的加速比为 $Sp(q)$, 则并行算法的效率 $Ep(q) = Sp(q) / q$ 。反映了并行系统中处理器的利用程度

可扩充性：其最简朴的含义是在确定的应用背景下，计算机系统（或算法或编程等）性能随处理器数的增加而按比例提高的能力。

在不同的约束条件下的加速比产生了三个重要的加速比性能定律：

Amdahl 定律：

约束条件：1，对于很多科学计算，实时性要求很高，即在此类应用中时间是个关键因素，而计算负载是固定不变的。为此在一定计算负载下，达到实时性可利用增加处理器数来提高计算速度。2，因为固定的计算负载是可分布在多个处理器上的，这样增加了处理器就加快了执行速度，从而达到了加速的目的。

$$S = \frac{W_s + W_p}{W_s + W_p / p}; \text{不带通信开销的计算公式}$$

$$s = \frac{W_x + W_p}{W_s + \frac{W_p}{p} + W_o} \text{带通信开销的计算公式}$$

Gustafson 定律：1，对于很多大型计算，精度要求很高，即此类应用中精度是个关键因素，而计算时间是固定不变的。此时为了提高精度，必须加大计算量，相应的亦必须增多处理器数目才能维持计算时间不变。2，除非学术研究，在实际应用中没有必要固定工作负载而使计算程序运行在不同数目的处理器上，增多处理器必须相应增大问题的规模才有实际意义。

$$S' = \frac{Ws + pW_p}{Ws + p * W_p / p} = \frac{Ws + pWp}{Ws + Wp} \text{(不考虑额外开销)}$$

$$S' = \frac{Ws + pWp}{Ws + Wp + Wo} \text{(考虑额外开销)}$$

Sun-Ni 定律：只要存储空间许可，应尽量增大问题的规模以产生更好的或更精确的解（此时可能使执行时间略有增加）。

$$S'' = \frac{fW + (1-f)G(p)W}{fW + (1-f)G(p)W / p} = \frac{f + (1-f)G(p)}{f + (1-f)G(p) / p} \text{(不考虑额外开销)}$$

$$S'' = \frac{fW + (1-f)G(p)W}{fW + (1-f)G(p)W / p + Wo} = \frac{f + (1-f)G(p)}{f + (1-f)G(p) / p + Wo / W} \text{(考虑额外开销)}$$

6、如何进行并行计算机性能评测？什么是基准测试程序？什么是可扩充性测量标准？

如何进行并行计算机性能评测：（没有如何进行并行计算性能评测）。

基准测试程序用于测试和预测计算机系统的性能，揭示了不同结构机器的长处和短处，为用户决定购买和使用哪种机器最适合他们的应用要求提供决策。

可扩充性测量标准：是在确定的应用背景下，计算机系统性能虽处理器数的增加而按比例提高的能力。

7、并行算法设计的一般过程 PCAM 是指什么？各个步骤中的主要判据是什么？

PCAM 是 Partitioning（划分）、Communication（通信）、Agglomeration（组合）和 Mapping（映射）首字母的拼写，它们代表了使用此法设计并行算法的四个阶段。上述各阶段简述如下：

划分：将整个计算分解成小的任务，其目的是尽量开拓并发执行的机会；

通信：确定诸任务执行中所需交换的数据和协调诸任务的执行，由此可检测上述划分的合理

性。

组合：按性能要求和实现的代价来考察前两阶段的结果，必要时可将一些小的任务组合成更大的任务以提高性能或减少通信开销。

映射：将每个任务分配到一个处理器上，其目的是最小化全局执行时间和通信成本以及最大化处理器的利用率。

划分判据：

- (1) 你所划分的任务数，是否至少高于目标机上处理器数目的一个量级？如果不是，则你在后继的设计步骤中将缺少灵活性。
- (2) 你的划分是否避免了冗余的计算和存储要求，如果不是，则所产生的算法对大型问题可能是不可扩放的。
- (3) 诸划分的任务是否尺寸大致相当，如果不是，则分配处理器时很难做到工作量均衡。
- (4) 划分的任务数是否与问题尺寸成比例？理想情况下，问题尺寸的增加应引起任务数的增加而不是任务尺寸的增加。如果不是这样，则你的算法可能不能求解更大的问题，尽管有更多的处理器。
- (5) 确认你是否采用了几种不同的划分法，多考虑几种选择可提高灵活性，同时要考虑域分解又要考虑功能分解。

通讯判据：

- (1) 所有任务是否均执行同样多的通信？如果不是，则所设计的算法的可扩展性可能是不好的。
- (2) 每个任务是否只与少许的近邻相通信？如果不是，则可能导致全局通信，在此情况下，应设法将全局通信结构化为局部通信结构。
- (3) 诸通信操作是否能并行执行？如果不是，则所设计的算法很可能是低效的和不可扩放的，在此情况下，设法试用分治技术来开发并行性。
- (4) 不同任务的计算能否并行执行？如果不是，则所设计的算法很可能是低效的和不可扩放的，在此情况下，可考虑重新安排通信/计算之次序等来改善之。

组合判定：

- (1) 用增加局部性方法施行组合是否减少了通信成本？如果不是，检查能否由别的组合策略来达到
- (2) 如果组合已造成重复计算，是否已权衡了其权益。
- (3) 如果组合已重复了数据，是否已证实这不会因限制问题尺寸和处理器数的变化范围而牺牲了可扩放性？
- (4) 由组合所产生的任务是否有类似的计算和通信代价。
- (5) 任务数目是否仍然与问题尺寸成比例？如果不是，算法则不是可扩放的。
- (6) 如果组合减少了并行执行的机会，是否已证实现在的并发性仍能适应目前和将来的并行机。
- (7) 在不导致负载不平衡，不增加软件工程代价和不减少可扩放性的前提下，任务数能否再进一步减少，在其他条件等同时，创建较少的大粒度的任务算法通常是简单有效的。
- (8) 如果并行化现有的串行程序，是否考虑了修改串行代码所增加的成本？如果此成本是高的，应考虑别的组合策略，它能增加代码重用的机会。

映射判据：

- (1) 如果采用集中式负载平衡方案，你是否已验证中央管理者不会成为瓶颈？
- (2) 如果采用动态负载平衡方案，你是否衡量过不同策略的成本？
- (3) 如果采用概率或循环法，你是否有足够多的任务来保证合理的负载平衡？典型地，

任务数应 10 倍于处理数。

(4) 如果要为一个复杂问题设计一个 SPMD 程序，你是否考虑过基于动态任务创建和消除的算法：后者可能得到一个更加简单的算法，但是性能可能有问题。

8、什么是域分解和功能分解？如何将全局通信转换为局部通信？什么是表面-容积效应和重复计算？映射的策略是什么？

域分解：也叫数据划分。所要划分的对象是些数据，这些数据可以是算法（或程序的输入数据，计算的输出数据，或者算法所产生的中间结果。

功能分解：也叫计算划分，其基本出发点不同于域分解，它首先关注于被执行的计算上，而不是计算所需的数据上，然后，如果所作的计算划分是成功的，再继续研究计算所需的数据上，这些数据可能是不相交的，这就意味着划分很成功，或者这些数据有相当的重叠，这就产生大量的通信，此时就暗示应考虑数据分解。

将全局通信转换为局部通信：

局部通信时，每个任务只与较少的几个近邻通信；全局通信中，每个任务与很多别的任务通信。并行算法可采用分治法进行优化一方面提高了算法的并行度；同时也可将全局通信转换为局部通信

表面-容积效应：一个任务的通信需求比例于它所操作的子域的表面积，而计算需求却比例于子域的容积。在一个二维问题中，“表面积”比例于问题的尺寸，而“容积”比例于问题尺寸的平方。因此一个计算单元的通信/计算之比随问题（任务）尺寸的增加而减少。

重复计算：它也称为冗余计算。有时候可以采用不必要的多余的计算的方法来减少通信要求和/或执行时间。

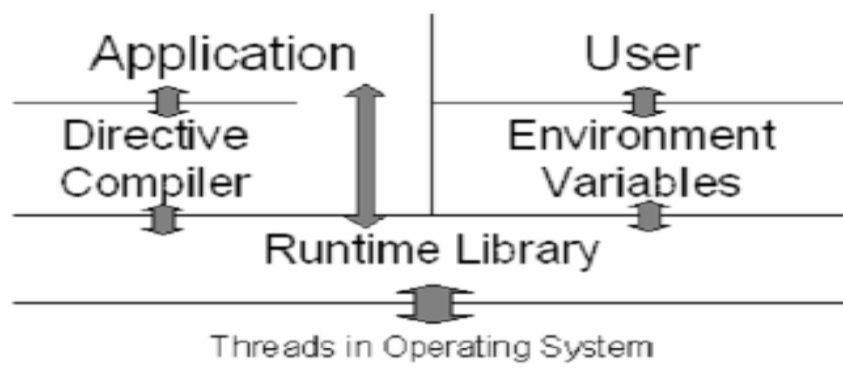
映射的策略：开发映射的主要目的是减少算法的总的执行时间，其策略有二：一是把那些能够并发执行的任务放在不同的处理器上以增强并行度；二是把那些需频繁通信的任务置于同一个处理器上以提高局部性。

9、什么是 OpenMP 的编程模型、体系结构、控制结构和数据域子句？

OpenMP 的编程模型

首先，OpenMP 是基于线程的并行编程模型（Parallel Programming Model），一个共享存储的进程由多个线程组成，OpenMP 就是基于已有线程的共享编程模型；其次，OpenMP 是一个外部的编程模型，而不是自动编程模型，它能够使程序员完全控制并行化。

体系结构



控制结构

并行域结构：一个并行域就是一个能够被多线程执行的程序块，它是最基本的 OpenMP 并行结构；**共享任务结构：**共享任务结构把其内封闭的代码段划分给线程队列中的各线程执行；**组合的并行共享任务结构；同步结构：**OpenMP 提供了多种同步结构来控制与其他线程相关的线程的执行。

数据域子句

OpenMP 的数据域子句用来定义变量的范围，它包括 PRIVATE、FIRSTPRIVATE、LASTPRIVATE、SHARED、DEFAULT、REDUCTION 和 COPYIN 等。

10、什么是 MPI 的消息、数据类型、通信域？如何应用 MPI 的扩展数据类型？

一个消息(message) 指在进程间进行的一次数据交换，在 MPI 中，一个消息由通信器、源地址、目的地址、消息标签和数据构成。

MPI 系统中数据的发送与接收操作都必须指定数据类型。数据类型可以是 MPI 系统预定义的，称为原始数据类型，也可以是用户在原始数据类型的基础上自己定义的数据类型。

MPI 中的通信域(Communicator)提供了一种组织和管理进程间通信的方法。MPI 通信域包括两部分进程组和通信上下文进程组即所有参加通信的进程的集合如果一共有 N 个进程参加通信则进程的编号从 0 到 N-1 通信上下文提供一个相对独立的通信区域不同的消息在不同的上下文中进行传递不同上下文的消息互不干涉通信上下文可以将不同的通信区别开来。

MPI 除了可以发送或接收连续的数据之外还可以处理不连续的数据其基本方法可采取允许用户自定义新的数据类型又称派生数据类型的方法，即可扩展的数据类型。

11、什么是 MPI 的阻塞通信和非阻塞通信？点到点通信模式有哪些？群集通信模式？

阻塞通信 (Blocking Communication)：当一个通信过程的请求的完成依赖与某个“事件”时，我们说它是阻塞式的。对于发送过程，数据必须被成功地发送或安全地拷贝到系统缓存以便于保存它的应用缓存可以被重用。对于接收操作，数据必须被安全地存储到接收缓存中以便它可以被利用。

非阻塞通信 (Nonblocking Communication)：当一个通信过程的完成不需要等待任何通信事件的完成就可以返回时，它就属于非阻塞通信，例如将数据从用户的内存拷贝到系统缓存或消息的到达。对于一个非阻塞发送，在它完成之后修改或重用应用缓存是很危险的，这就需要编程者保证此时应用缓存是空闲的可以被重用。

点到点通信有四种通信模式

标准通信模式：MPI_SEND MPI_RECV MPI_ISEND MPI_Irecv

缓冲通信模式：MPI_BSEND MPI_IBSEND

同步通信模式：MPI_SSEND MPI_ISSend

就绪通信模式：MPI_RSEND MPI_IRSEND

集群通信 (Collective Communication)：集群通信的一个最特征是指定的 Communicator 中所有的进程都参与通信操作，它是点到点通信的扩展。包括 1) 路障：同步所有进程 2) 广播：从一个进程发送一条数据给所有进程 3) 收集：从所有进程收集数据到另一个进程 4) 散播：从一个进程散发多条数据给所有进程 5) 归约：包括求和、求积等。

12、编写 OpenMP 和 MPI 计算 π 的代码